

PYTHON PROGRAMMING

DATA TYPES

DATA TYPES

Every value belongs to a specific data type in Python. Data type identifies the type of data values a variable can hold and the operations that can be performed on that data. Figure 5.6 enlists the data types available in Python.

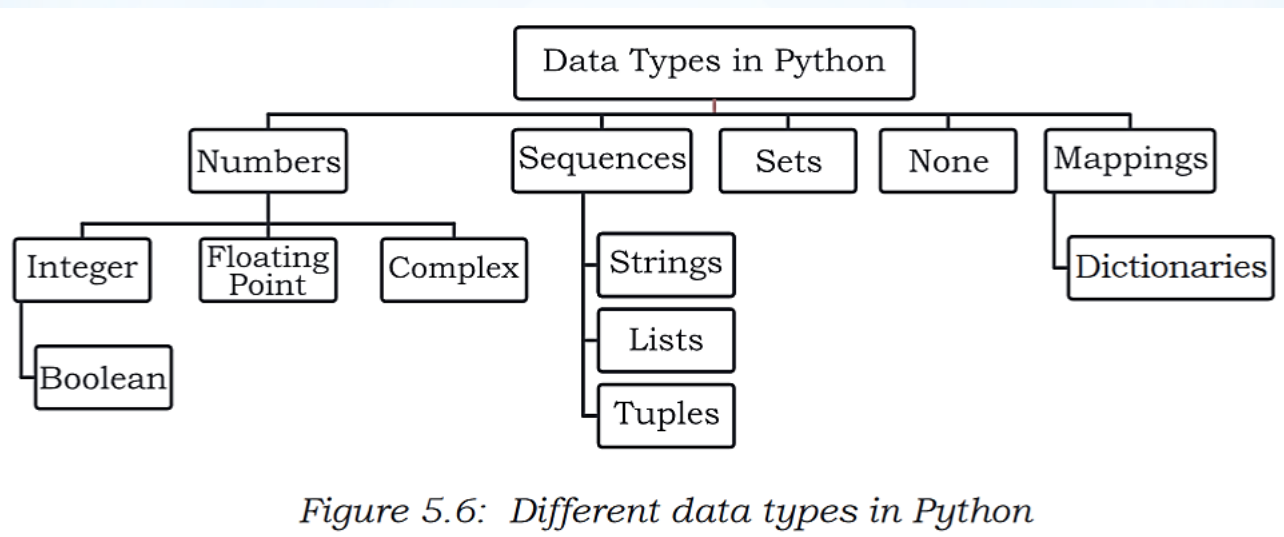


Figure 5.6: Different data types in Python

Number

Number data type stores numerical values only. It is further classified into three different types: `int`, `float` and `complex`.

Table 5.2 Numeric data types

Type/ Class	Description	Examples
<code>int</code>	integer numbers	-12, -3, 0, 125, 2
<code>float</code>	real or floating point numbers	-2.04, 4.0, 14.23
<code>complex</code>	complex numbers	$3 + 4j$, $2 - 2j$

Boolean data type (`bool`) is a subtype of integer. It is a unique data type, consisting of two constants, `True` and `False`. Boolean `True` value is non-zero, non-null and non-empty. Boolean `False` is the value zero.

Let us now try to execute few statements in interactive mode to determine the data type of the variable using built-in function `type()`.

Example 5.3

```
>>> num1 = 10
>>> type(num1)
<class 'int'>
```

```
>>> num2 = -1210
>>> type(num2)
<class 'int'>
```

```
>>> var1 = True
>>> type(var1)
<class 'bool'>
```

```
>>> float1 = -1921.9
>>> type(float1)
<class 'float'>
>>> float2 = -9.8*10**2
>>> print(float2, type(float2))
-980.00000000000001 <class 'float'>

>>> var2 = -3+7.2j
>>> print(var2, type(var2))
(-3+7.2j) <class 'complex'>
```

Variables of simple data types like integers, float, boolean, etc., hold single values. But such variables are not useful to hold a long list of information, for example, names of the months in a year, names of students in a class, names and numbers in a phone book or the list of artefacts in a museum. For this, Python provides data types like tuples, lists, dictionaries and sets.

Sequence

A Python sequence is an ordered collection of items, where each item is indexed by an integer. The three types of sequence data types available in Python are Strings, Lists and Tuples.

(A) String

String is a group of characters. These characters may be alphabets, digits or special characters including spaces. String values are enclosed either in single quotation marks (e.g., 'Hello') or in double quotation marks (e.g., "Hello"). The quotes are not a part of the string, they are used to mark the beginning and end of the string for the interpreter. For example,

```
>>> str1 = 'Hello Friend'  
>>> str2 = "452"
```

We cannot perform numerical operations on strings, even when the string contains a numeric value, as in str2.

(B) List

List is a sequence of items separated by commas and the items are enclosed in square brackets [].

Example 5.4

```
#To create a list
>>> list1 = [5, 3.4, "New Delhi", "20C", 45]
#print the elements of the list list1
>>> print(list1)
[5, 3.4, 'New Delhi', '20C', 45]
```

(C) Tuple

Tuple is a sequence of items separated by commas and items are enclosed in parenthesis (). This is unlike list, where values are enclosed in brackets []. Once created, we cannot change the tuple.

Example 5.5

```
#create a tuple tuple1
>>> tuple1 = (10, 20, "Apple", 3.4, 'a')
#print the elements of the tuple tuple1
>>> print(tuple1)
(10, 20, "Apple", 3.4, 'a')
```

Set

Set is an unordered collection of items separated by commas and the items are enclosed in curly brackets { }. A set is similar to list, except that it cannot have duplicate entries. Once created, elements of a set cannot be changed.

Example 5.6

```
#create a set
>>> set1 = {10,20,3.14,"New Delhi"}
>>> print(type(set1))
<class 'set'>
>>> print(set1)
{10, 20, 3.14, "New Delhi"}
#duplicate elements are not included in set
>>> set2 = {1,2,1,3}
>>> print(set2)
{1, 2, 3}
```

None

None is a special data type with a single value. It is used to signify the absence of value in a situation. None supports no special operations, and it is neither same as False nor 0 (zero).

Example 5.7

```
>>> myVar = None
>>> print(type(myVar))
<class 'NoneType'>
>>> print(myVar)
None
```


Mapping

Mapping is an unordered data type in Python. Currently, there is only one standard mapping data type in Python called dictionary.

(A) Dictionary

Dictionary in Python holds data items in key-value pairs. Items in a dictionary are enclosed in curly brackets { }. Dictionaries permit faster access to data. Every key is separated from its value using a colon (:) sign. The key : value pairs of a dictionary can be accessed using the key. The keys are usually strings and their values can be any data type. In order to access any value in the dictionary, we have to specify its key in square brackets [].

Example 5.8

```
#create a dictionary
>>> dict1 = {'Fruit':'Apple',
             'Climate':'Cold', 'Price(kg)':120}
>>> print(dict1)
{'Fruit': 'Apple', 'Climate': 'Cold',
 'Price(kg)': 120}
>>> print(dict1['Price(kg)'])
120
```

❖ Dictionary is a collection of key-value pairs.

❖ Characteristics of Dictionary:

- Unordered
- Mutable
- Indexed
- **Cannot hold duplicate keys**

****As of Python version 3.7, dictionaries are ordered. In Python 3.6 and earlier, dictionaries are unordered.**

Python Collections (Arrays)

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- **Dictionary** is a collection which is ordered** and changeable. No duplicate members.

*Set items are unchangeable, but you can remove and/or add items whenever you like.

**As of Python version 3.7, dictionaries are ordered. In Python 3.6 and earlier, dictionaries are unordered.

In Python 3.6 and earlier, dictionaries are unordered. When we say that dictionaries are ordered, it means that the items have a defined order, and that order will not change. Unordered means that the items does not have a defined order, you cannot refer to an item by using an index.

****As of Python version 3.7, dictionaries are ordered. In Python 3.6 and earlier, dictionaries are unordered.**

In the context of Object Oriented Programming (OOP), objects are a representation of the real world, such as employee, student, vehicle, box, book, etc. In any object oriented programming language like C++, JAVA, etc., each object has two things associated with it: (i) data or attributes and (ii) behaviour or methods. Further there are concepts of class and class hierarchies from which objects can be instantiated. _____,

Python also comes under the category of object oriented programming. However, in Python, the definition of object is loosely casted as some objects may not have attributes or others may not have methods.

EVERYTHING IS AN OBJECT

Python treats every value or data item whether numeric, string, or other type as an object in the sense that it can be assigned to some variable or can be passed to a function as an argument.

Every object in Python is assigned a unique identity (ID) which remains the same for the lifetime of that object. This ID is akin to the memory address of the object. The function id() returns the identity of an object.

Example 5.2

```
>>> num1 = 20
>>> id(num1)
1433920576          #identity of num1
>>> num2 = 30 - 10
>>> id(num2)
1433920576          #identity of num2 and num1
                        #are same as both
refers to           #object 20
```

Mutable and Immutable Data Types

- ❖ Sometimes we may require to change or update the values of certain variables used in a program.
- ❖ However for certain data types, Python does not allow us to change the values once a variable of that type has been created and assigned values
- ❖ Variables whose values **can be changed** after they are created and assigned are called **mutable**.
- ❖ Variables whose values **cannot be changed** after they are created and assigned are called **immutable**.

When an attempt is made to update the value of an immutable variable, the old variable is destroyed and a new variable is created by the same name in the memory.

Python data types can be classified into mutable and immutable as shown in Figure 5.7.

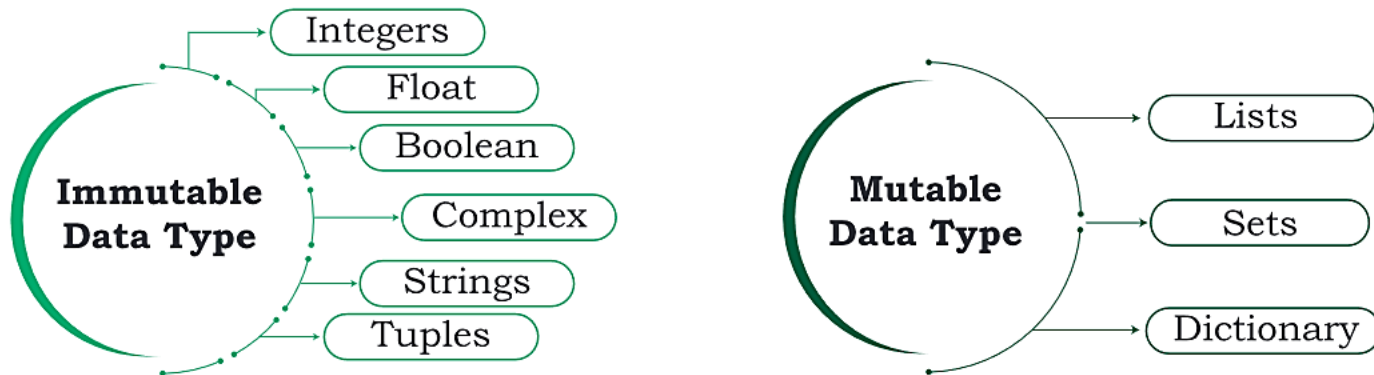
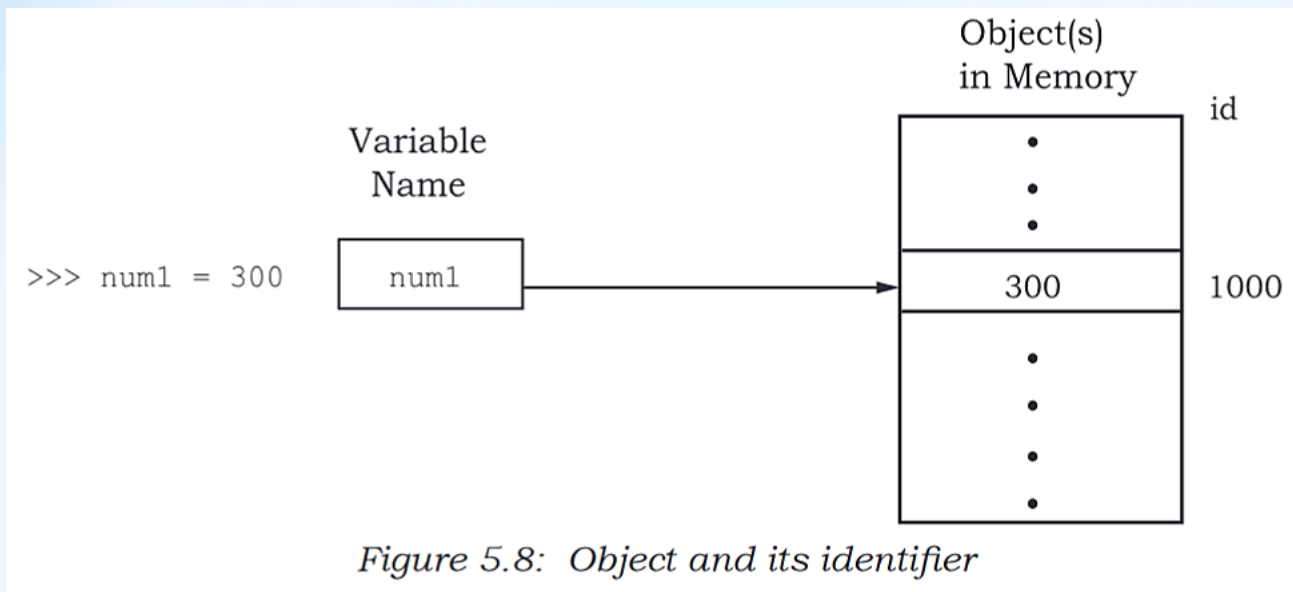


Figure 5.7: Classification of data types

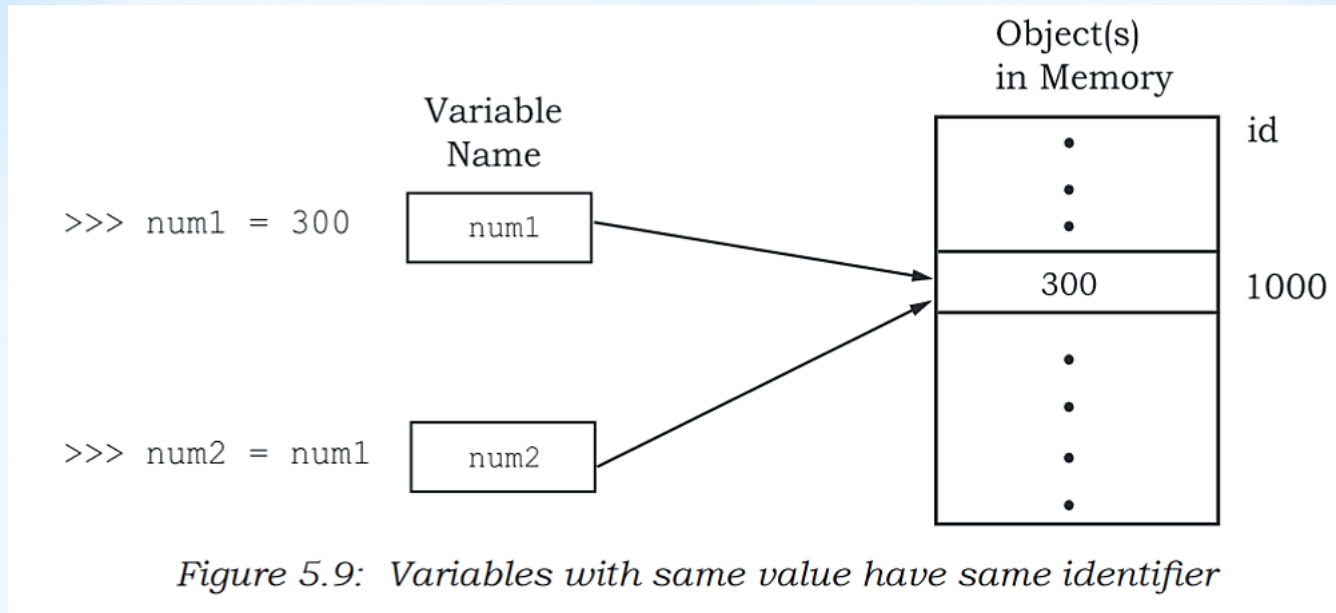


Let us now see what happens when an attempt is made to update the value of a variable.

```
>>> num1 = 300
```

This statement will create an object with value 300 and the object is referenced by the identifier num1.

```
>>> num2 = num1
```



The statement `num2 = num1` will make `num2` refer to the value 300, also being referred by `num1`, and stored at memory location number, say 1000. So, `num1` shares the referenced location with `num2`.

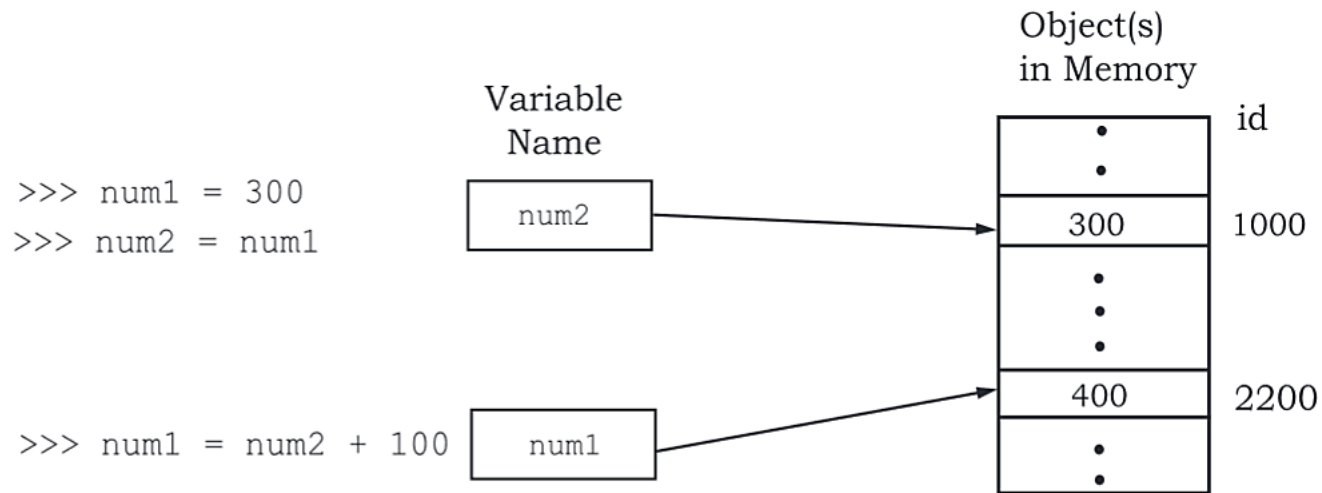


Figure 5.10: Variables with different values have different identifiers

In this manner Python makes the assignment effective by copying only the reference, and not the data:

```
>>> num1 = num2 + 100
```

This statement `num1 = num2 + 100` links the variable `num1` to a new object stored at memory location number say 2200 having a value 400. As `num1` is an integer, which is an immutable type, it is rebuilt,

Deciding Usage of Python Data Types

- ❖ It is preferred to use the **List** when we need a simple iterable collection of data that may go for frequent modifications.
For example, if we store the names of students of a class in a list, then it is easy to update the list when some new students join or some leave the course.
- ❖ **Tuples** are used when we do not need any change in the data.
For example, names of months in a year.
- ❖ When we need uniqueness of elements and to avoid duplicacy it is preferable to use **Sets**.
For example, list of artefacts in a museum.
- ❖ If our data is being constantly modified or we need a fast lookup based on a custom key or we need a logical association between the key : value pair, it is advised to use **Dictionaries**.
For example, A mobile phone book is a good application of dictionary.